OPTIGA™ TPM User Guide

OPTIGA™ TPM SLx 967y TPM2.0 on Raspberry Pi® 4 QuickStarter User Guide

Devices

- OPTIGA™ TPM SLB 9672 TPM2.0 FW 15.xx
- OPTIGA™ TPM SLB 9672 TPM2.0 FW 16.xx
- OPTIGA™ TPM SLB 9673 TPM2.0
- OPTIGA™ TPM SLB 9670 TPM2.0
- OPTIGA™ TPM SLI 9670 TPM2.0
- OPTIGA™ TPM SLM 9670 TPM2.0

About this document

Scope and purpose

This document explains how the OPTIGA™ TPM SLx 967y TPM2.0 can be used on a Raspberry Pi® 4. The same setup also works and has been tested on a Raspberry Pi® 3. However, in this document we refer to "Raspberry Pi® 4". The Linux kernel described in this document is version 5.15 (the latest kernel in Raspberry Pi® OS) as well as version 6.0 (the latest upstream kernel). These kernels have integrated TPM 2.0 driver support, which simplifies TPM setup with the Raspberry Pi® 4. This User Guide shows how fast and simple the TPM 2.0 setup is, even with basic Linux knowledge.

Intended audience

This document is intended for customers who want to increase the security level of their embedded platforms using a TPM 2.0 and like to evaluate how to start with the TPM software integration for their target applications.

Table of contents

1 About this document **3**
1.1 Scope and purpose . . . 3
1.2 Evaluation Boards . . . 4
1.3 Intended audience . . . 4

2 Introduction **4**
2.1 Motivation . . . 4
2.2 Scope . . . 5
2.3 Acronyms and Abbreviations . . . 6

3 Hardware Setup **6**
3.1 Infineon OPTIGA™ TPM SLB 9672 RPi Evaluation Boards . . . 7
3.2 Infineon OPTIGA™ TPM SLB 9673 RPi Evaluation Boards . . . 7

4 OS and Driver Setup **8**
4.1 Raspberry Pi® OS . . . 8

5 Software Setup **9**
5.1 Install the OpenSource TPM Software Stack `tpm2-tss`, TPM2 tools and application integrations . . . 9

6 Using the TPM with tpm2-tss and tpm2-tools **10**
6.1 Introduction . . . 10
6.2 Verifying TSS2 functionality . . . 11
6.3 Using TPM2.0 Tools . . . 11

7 Using TPM with OpenSSL **17**
7.1 Installing the software . . . 17
7.2 Interfacing with OpenSSL via command line . . . 18
7.3 Interfacing with OpenSSL from *NGINX* WebServer . . . 19
7.4 Interfacing with OpenSSL from *curl* WebClient . . . 23

8 Using TPM with PKCS11 **25**

9 Using TPM for HDD encryption **29**

10 Appendix **32**
10.1 Troubleshooting . . . 32
10.2 Support on other Linux® distributions . . . 35
10.3 Compiling Linux® with TPM2 driver (for Raspberry Pi® OS) . . . 39
10.4 Compiling the TSS from source . . . 42
10.5 Accessing the TPM with eltt2 . . . 43
10.6 Working with the Raspberry Pi® . . . 43

1 About this document

1.1 Scope and purpose

This document explains how the OPTIGA™ TPM SLx 967y TPM2.0 can be used on a Raspberry Pi® 4. The same setup also works and has been tested on a Raspberry Pi® 3. However, in this document we refer to "Raspberry Pi® 4". The Linux® kernel described in this document is version 5.15 (the latest kernel in Raspberry Pi® OS) as well as version 6.0 (the latest upstream kernel). These kernels have integrated TPM 2.0 driver support, which simplifies TPM setup with the Raspberry Pi® 4. This Application Note shows how fast and simple the TPM 2.0 setup is, even with basic Linux® knowledge.

The OPTIGA™ TPM SLx 967y TPM2.0 uses an SPI or I^2C interface to communicate with the Raspberry Pi®. The OPTIGA™ TPM SLx 967y TPM2.0 product family with SPI interface consists of 6 different products:

- OPTIGA™ TPM SLB 9672 TPM2.0 FW 15.xx standard security applications
- OPTIGA™ TPM SLB 9672 TPM2.0 FW 16.xx embedded security applications using SPI bus
- OPTIGA™ TPM SLB 9673 TPM2.0 embedded security applications using I^2C bus
- OPTIGA™ TPM SLB 9670 TPM2.0 standard security applications
- OPTIGA™ TPM SLI 9670 TPM2.0 automotive qualified security applications
- OPTIGA™ TPM SLM 9670 TPM2.0 industrial qualified security applications

We refer with "OPTIGA™ TPM SLx 967y TPM2.0" to all of the above 6 members of the OPTIGA™ TPM 2.0 product family.

OPTIGA™ TPM SLx 967y TPM2.0 products are fully TCG compliant TPM products with CC (EAL4+) and FIPS certification. The OPTIGA™ TPM SLx 967y TPM2.0 products "standard", "automotive" and "industrial" differ with regards to supported temperature range, lifetime, quality grades, test environment, qualification and reliability to fit the target applications requirements. The OPTIGA™ TPM SLx 967y TPM2.0 products "embedded security" provide additional functionality. The OPTIGA™ TPM SLB 9672 and 9673 can also be ordered for various operating temperature ranges. For more details refer to Infineon's website[1]. Since all OPTIGA™ TPM SLx 967y TPM2.0 products are fully TCG compliant with regards to the SPI/I^2C and software interface, the described steps to integrate a TPM 2.0 in a Raspberry Pi® 4 Linux® environment are the same and valid for all variants. We are referring in this document also to the OPTIGA™ TPM SLx 967y TPM2.0 by using simply "TPM". An overview of all Infineon OPTIGA™ TPM products can be found on Infineon's website[2]. More information about the TPM in general and how to integrate it into a platform can be found in the corresponding specifications of the Trusted Computing Group (TCG)[3].

The described steps to integrate an OPTIGA™ TPM in a Raspberry Pi® 4 Linux® environment can be performed with one of the Infineon Evaluation Boards for SLx 967y TPM2.0, listed in the following Table.

[1] https://www.infineon.com/tpm
[2] https://www.infineon.com/tpm
[3] https://trustedcomputinggroup.org/resource/tpm-main-specification/

1.2 Evaluation Boards

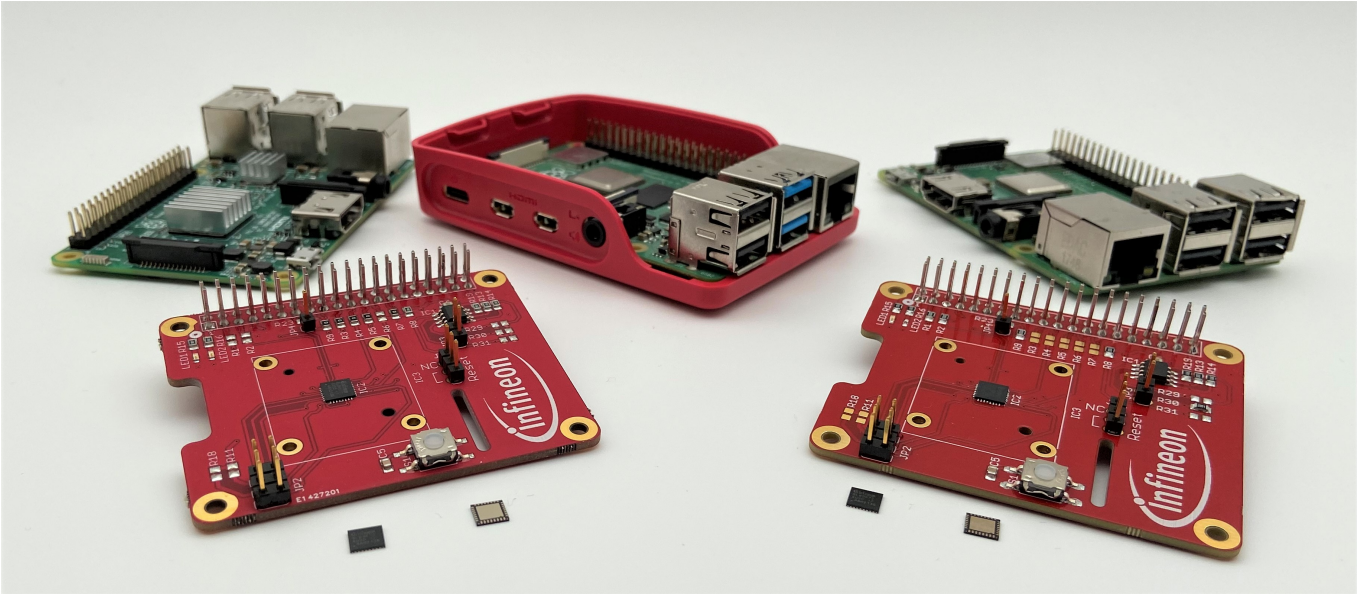
Supported TPM	Order type	Order number
OPTIGA™ TPM SLB 9672 TPM2.0 FW16.23	OPTIGA™ TPM SLB 9672 RPI evaluation board	SP006005644
OPTIGA™ TPM SLB 9673 TPM2.0 FW26.13	OPTIGA™ TPM SLB 9673 RPI evaluation board	SP006005648
OPTIGA™ TPM SLB 9670 TPM2.0	IRIDIUM SLB 9670 TPM2.0	SP001596592
OPTIGA™ TPM SLI 9670 TPM2.0	IRIDIUM SLI 9670 TPM2.0	SP004232000
OPTIGA™ TPM SLM 9670 TPM2.0	IRIDIUM SLM 9670 TPM2.0	SP004232004

These 5 Infineon Boards are referred in the following as “Infineon SLx 967y TPM2.0 Evaluation Board”

1.3 Intended audience

This document is intended for customers who want to increase the security level of their embedded platforms using a TPM 2.0 and like to evaluate how to start with the TPM software integration for their target applications.

2 Introduction



2.1 Motivation

Two of the basic principles of information security when exchanging data are confidentiality and authenticity. While the first one is obvious to almost everyone, the second one often does not get the appropriate amount of attention. Authenticity is just as important as confidentiality, because for example the highest confidentiality of secret data is worth nothing in case the secret data come from or is shared with the wrong communication partner.

A TPM adds additional security features to a platform, which will not only help in gaining confidentiality but also authenticity of exchanged data or a communication partner.

One of the most important features to achieve these goals is that sensitive data such as cryptographic keys or secrets can be stored inside the TPM, where the data is protected by the TPM's hardware from unauthorized access or manipulation. This allows confidentiality of the data on a high level. But in addition to this protection, the data can also be bound to a single TPM and thus to the platform hosting this TPM. This in turn can be used to bring in another authenticity factor, since only the platform or user operating the correct TPM is able to provide the correct authentication secret required to access the protected sensitive data.

Another basic feature of the TPM is that it can function as a starting point for establishing a root of trust, which allows the detection of unauthorized modifications to a platform's hardware or software.

Additionally, the TPM can be used to perform several cryptographic operations on data in hardware, decreasing the vulnerability to several kinds of attacks, such as reading out unencrypted sensitive data from the platform's memory.

2.2 Scope

This document is intended for users to help them getting familiar how to use a subset of the TPM's functionalities on embedded platforms running Linux. The final goal in this document is to interact with the TPM 2.0 to be able to run the test scripts provided in the tpm2-software GitHub community[4].

The topics described in this document are:

- The Evaluation Board with its expansion header for the Raspberry Pi® 4 and 5[5].
- Usage of a TPM 2.0 on a Raspberry Pi® 4 and 5.

This document has been generated using Raspberry Pi® OS version "bookworm" with Linux® kernel version 6.1.0-rpi4-rpi-v8.

In this application note we use Raspberry Pi® OS, a Raspberry Pi® 4 Linux® distribution based on Debian® to describe the system setup and the application of the TPM 2.0. If not explicitly said otherwise, the application note refers to specific versions of the required Linux® packages to prevent compatibility problems. The mentioned versions were publicly available and downloaded at the time of release from the specified links and repositories.

[4] https://github.com/tpm2-software
[5] https://www.raspberrypi.org/

2.3 Acronyms and Abbreviations

Acronym	Explanation
DTS	Device Tree Source
DTB	Device Tree Blob
DTBO	Device Tree Blob Overlay
I^2C	Inter-Integrated Circuit
OS	Operating System
PC	Personal Computer
RC	Release Candidate
RPi	Raspberry Pi®
SoC	System on a Chip
SPI	Serial Peripheral Interface
SRK	Storage Root Key
SSH	Secure Shell
TCG	Trusted Computing Group
TPM	Trusted Platform Module
TSS	TPM Software Stack

3 Hardware Setup

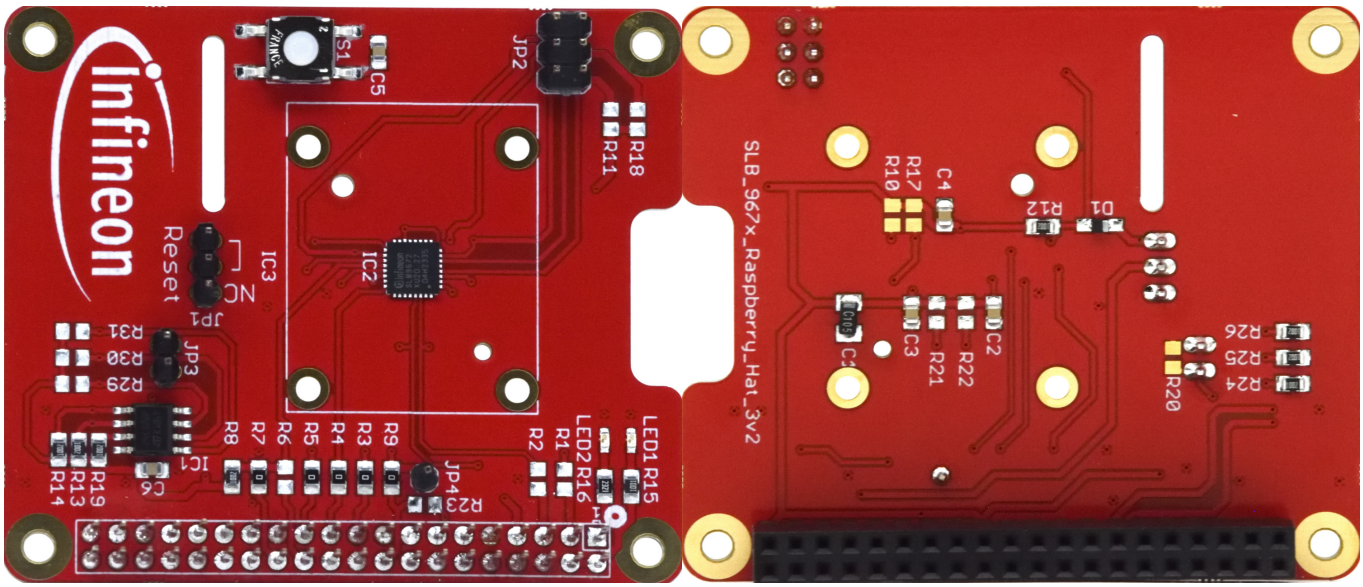
The required hardware to perform the steps described in this document consists of:

- Developer PC: This platform is used for installing the Rapsberry Pi® OS image following the steps provided in this guide[6]. (Note: The advanced menu of the Rapsberry Pi® Imager program now offers the setting of a default user and password, since the default user “pi” and password “raspberry” have been disabled.).
- Infineon OPTIGA™ TPM SLB 967y TPM2.0 RPI Evaluation Board or IRIDIUM SLx 9670 board. This board contains the Infineon OPTIGA™ TPM SLx 967y TPM2.0 mounted on an easy-to-use hardware board, which can be attached to the Raspberry Pi® 4.
- Raspberry Pi® 4:
  - Raspberry Pi® 4 board
  - Micro SD Card with at least 8 GB
  - USB-C USB cable and power supply
- MicroSD-Card Reader (for the developer PC)

[6] https://www.raspberrypi.org/documentation/installation/installing-images/README.md

3.1 Infineon OPTIGA™ TPM SLB 9672 RPi Evaluation Boards

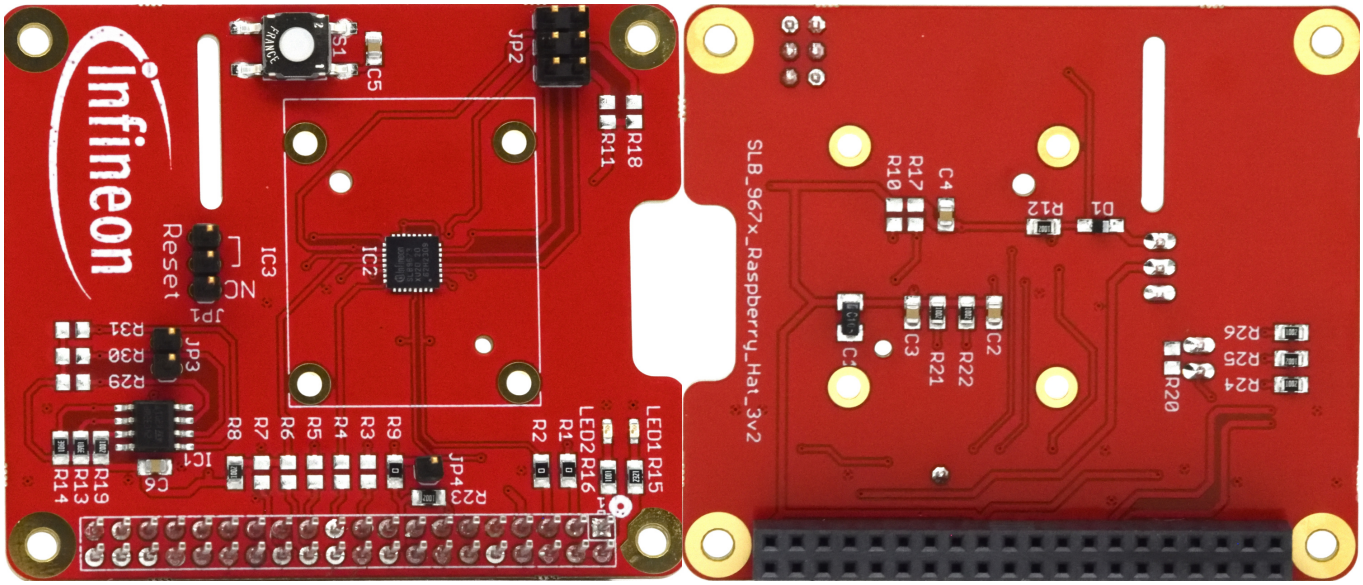
The Infineon OPTIGA™ TPM SLB 9672 TPM2.0 SPI RPi Evaluation Board can be connected to a Raspberry Pi® 4 via its extension header. For data transfer the board uses the SPI bus:



In order to distinguish this board from the OPTIGA™ TPM SLB 9673 Evaluation board, note that the resistors R1 and R2 are clear, but R3, R4, R5 and R7 are connected.

3.2 Infineon OPTIGA™ TPM SLB 9673 RPi Evaluation Boards

The Infineon OPTIGA™ TPM SLB 9673 TPM2.0 RPi Evaluation I^2C Board can be connected to a Raspberry Pi® 4 via its extension header. For data transfer the board uses the I^2C bus:



In order to distinguish this board from the OPTIGA™ TPM SLB 9672 Evaluation board, note that the resistors R1 and R2 are connected, but R3, R4, R5 and R7 are clear.

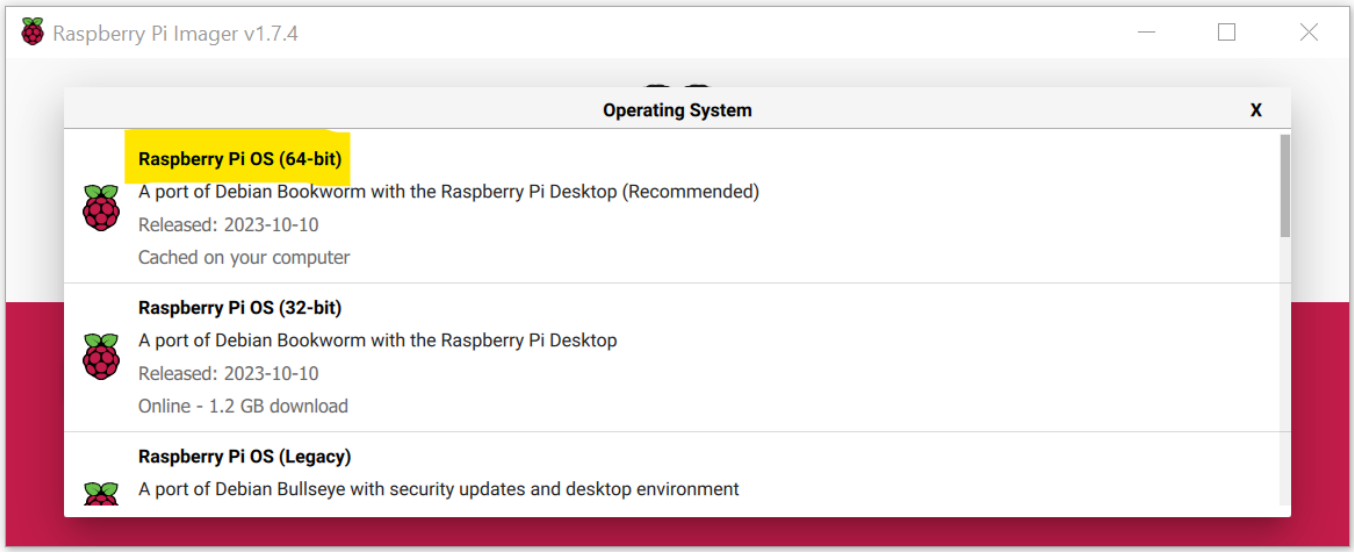
4 OS and Driver Setup

4.1 Raspberry Pi® OS

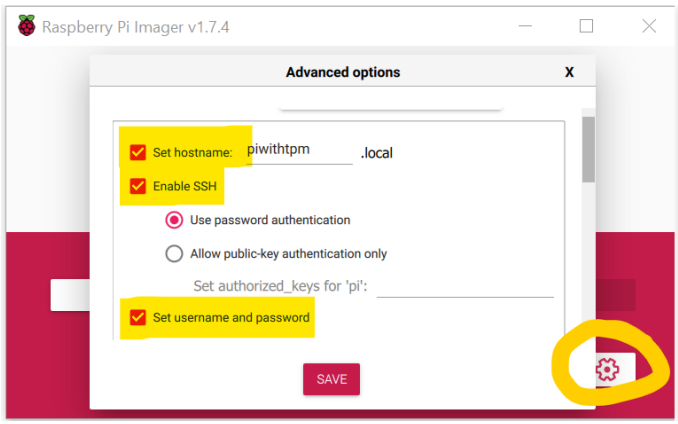
Flashing Raspberry Pi® OS

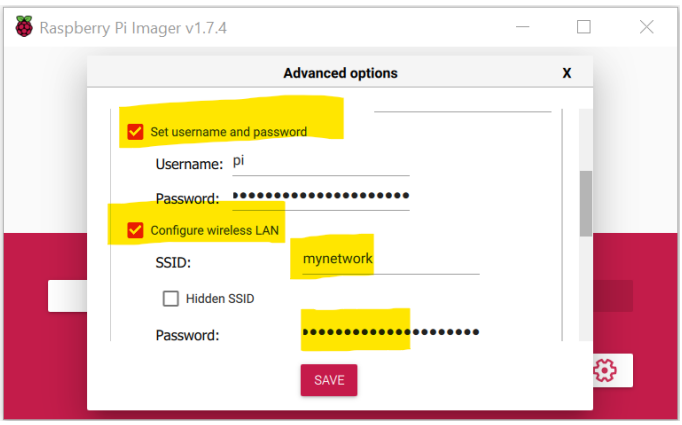
Download Raspberry Pi® Imager[7] and install and start the application.

Choose the Raspberry Pi® OS image:



Visit the advanced options menu on the bottom right of the window. Configure the hostname, username and password, allow ssh access and configure wifi credentials:





[7] https://www.raspberrypi.com/software/

Optiga™ TPM SLB 9672 and SLB 9673 RPi Evaluation Boards

Insert the flashed SD card and boot the Raspberry Pi® 4. The driver should be automatically loaded and configured. After boot, you can check for the existence of `/dev/tpm0` and `/dev/tpmrm0`:

```
ls -l /dev/tpm*
```

During the boot process the Linux® kernel is configured using the device tree information of the device how the host CPU and all its peripherals are connected. These device tree information is static for the basic board, but can also be extended using so-called device tree overlays. The Optiga™ TPM SLB967y RPi Evaluation boards contain an EEPROM chip that contain these device tree overlays. The Raspberry Pi® OS Linux® kernel is configured in such a way, that it reads out the EEPROM auf such boards and applies the configuration automatically.

Note: on older Optiga™ TPM SLx 9670 Iridium board, you need to manually enable the driver. After flashing, remove and reinsert the SD card into your PC. Edit the file `config.txt` from the FAT32 partition labeled `boot`. Add the following line to the `[all]` section:

```
dtoverlay=tpm-slb9670
```

5 Software Setup

5.1 Install the OpenSource TPM Software Stack `tpm2-tss`, TPM2 tools and application integrations

The tpm2-software community provides an Open-Source TPM Software Stack (TSS) implementation as well as a variety of further tools.

In order to test this, run the following commands:

```
sudo apt --yes install libtss2-* tpm-udev tpm2-abrmd tpm2-tools

# Add the current user to group tss (with privilege to access the TPM)
sudo usermod --append --groups tss $(whoami)

# Reboot for group change to become effective
sudo reboot
```

Now test the connection to TPM

```
tpm2 getcap properties-fixed
```

The output should look similar to this:

```
$ tpm2 getcap properties-fixed
TPM2_PT_FAMILY_INDICATOR:
  raw: 0x322E3000
  value: "2.0"
TPM2_PT_LEVEL:
  raw: 0
TPM2_PT_REVISION:
  raw: 0x9F
  value: 1.59
```

```
TPM2_PT_DAY_OF_YEAR:
  raw: 0xAA
TPM2_PT_YEAR:
  raw: 0x7E4
TPM2_PT_MANUFACTURER:
  raw: 0x49465800
  value: "IFX"
TPM2_PT_VENDOR_STRING_1:
  raw: 0x534C4239
  value: "SLB9"
TPM2_PT_VENDOR_STRING_2:
  raw: 0x36373200
  value: "672"
...
```

6 Using the TPM with tpm2-tss and tpm2-tools

The following sections describe how to start and use the TPM 2.0 on the Raspberry Pi® with `tpm2-tss` and `tpm2-tools`. Be sure that you have successfully completed all setup steps as described in the previous sections. For general information about the TPM, its operational states and requirements see the TPM specification[8]. All of the steps in this section have to be executed on the Raspberry Pi® (unless described otherwise). There are several ways to interact with the Raspberry Pi®, see section 6.8 for more information.

6.1 Introduction

`tpm2-tss` is a TPM Software Stack (TSS) for the TPM 2.0 and consists of the software stack libraries. It provides several functionalities, for example:

- Simplifies the execution of TPM commands by generating and processing TPM data structures by utilizing cryptographic functions.
- Marshals and unmarshals TPM commands to / from raw byte streams.
- Supports the establishment and management of secure communication from the host processor to the TPM chip with transport sessions.

`tpm2-abrmd` is the TPM Access Broker and Resource Manager Daemon and its purpose is to

- Provides resource management for the TPM, for example regarding sessions or the generation and storage of cryptographic keys.
- Multiplexes access to the TPM, so that different applications and / or separate users can independently use the TPM functions. `/dev/tpmrm0` a basic set of these operations is also provided by the Linux® kernel TPM driver. The limitation is that it does not work well with long-lived authorization sessions because it does not support session context ungaping.

`tpm2-tools` is a set of executables, which can be used to issue TPM 2.0 commands from the user shell. The tpm2-tools call the corresponding TSS2 function, which send commands to the resource manager. It converts these commands to raw bytes and sends them to the TPM driver in the kernel.

[8] https://trustedcomputinggroup.org/resource/tpm-library-specification/

A non-comprehensive list of software support can also be found on the tpm2-software community website[9].

6.2 Verifying TSS2 functionality

In case all steps in this document were performed correctly so far, the TSS2 and tools should work now. You can easily test this by executing the following command:

```
tpm2 getcap properties-fixed
```

In case this does not work, please refer to the *Troubleshooting section* (page 32) in the appendix of this document.

6.3 Using TPM2.0 Tools

A complete overview of all tpm2.0-tools can be seen in `man tpm2`[10]. It shows explanations for every command and its parameters. Further, the list of commands and documentation on each command can be retrieved from the tools manpages directly:

```
man tpm2
man tpm2 getcap
```

Encrypt and Decrypt a File using RSA 2048

Using asymmetric cryptography to securely exchange data between two peers is the basis for a lot of applications. The public key can be distributed to anyone who wants to send encrypted data to the owner of the private key. Usually, only small portions of data are encrypted by an asymmetric cryptographic algorithm because it takes significant amount of CPU power to do the mathematical calculations. These small data can be a symmetric key which is used in a later step for bulk encryption or configuration data for IoT applications. This section shows how a file can be encrypted and decrypted using the TPM 2.0 which is plugged to a RPi. In a real world scenario, the encryption may be done on a remote device which wants to send confidential data to the RPi. The encryption can be done on non-secure hardware as well because there is no data involved in the encryption process (plain-text, public key and cipher-text) which can be used to get any information about the private key. Therefore, using the TPM as an encryption service is possible, but not necessary.

The commands which are needed to execute the encryption and decryption use case are:

- tpm2 createprimary
- tpm2 create
- (tpm2 loadexternal)
- (tpm2 rsaencrypt)
- tpm2 load
- tpm2 rsadecrypt

[9] https://tpm2-software.github.io/software
[10] https://github.com/tpm2-software/tpm2-tools/blob/master/man/tpm2.1.md#arguments

The command tpm2 createprimary creates and loads a Primary Object under one of the Primary Seeds. To specify which seed should be used, the parameter `--hierarchy` has to be either o: Owner, p: Platform, e:Endorsement, n: Null. In this example, the Primary Object is created in the Endorsement Hierarchy. The command line option `--hash-algorithm` defines the hash algorithm used for computing the name of the object (0x000B: SHA256), `--key-algorithm` defines the algorithm associated with this object (0x0001: RSA) and `--context` defines the file to save the object context (primary_ctx). In the code below the hierarchy, hash algorithm and the object algorithm are provided as input and the object reference (–context primary_ctx) is returned by the tpm2 createprimary function. This reference is then used in subsequent code an input.

```
tpm2 createprimary --hierarchy e --hash-algorithm sha256 --key-algorithm rsa2048 --key-context primary_ctx
# Output file: primary_ctx
```

The tpm2 create command creates an encryption and decryption key and returns both the public and private portion of the key to the application . Both parts are stored as files (`key_pub`, `key_priv`) in the folder where the command was executed. The private part is encrypted with the public part of the parent key and can therefore only be decrypted and used inside the TPM.

```
tpm2 create --hash-algorithm sha256 --key-algorithm rsa2048 --parent-context primary_ctx --public key_pub --private key_priv
# Input file: primary_ctx - Output files: key_pub, key_priv
```

In order to encrypt, only the public key is required and thus can be loaded on any TPM without a password and without reference to the parent key of the private key. Due to resource limitations of the TPM, the keys are stored outside the chip, the public part unencrypted, the private part encrypted. This allows creating and storing an unlimited amount of keys with a single TPM 2.0 without any security drawback. The only disadvantage is that every key which shall be used for a cryptographic operation must be loaded into the TPM in advance of the actual operation. This mechanism can be simplified and accelerated using persistent storage inside the TPM for frequently used keys. Since performance is not a requirement for this sample use case, the keys are not stored permanently inside the TPM and must be loaded beforehand. The encryption key (`key_pub`) is loaded inside the TPM using the command below. It needs to have a parameter defining the hierarchy where the encryption key should be loaded, the public key itself and returns the reference which will be used in the subsequent encryption process:

```
tpm2 loadexternal --hierarchy n --public key_pub --key-context rsaencrypt_key_ctx
# Input file: key_pub - Output file: rsaencrypt_key_ctx
```

To encrypt data with the TPM, a filename for the plain-text input data (`plaintext_data`) and a filename for the encrypted data (`cipher_data`) must be provided.

```
echo "Hello World" >plaintext_data
# Output file: plaintext_data

tpm2 rsaencrypt --key-context rsaencrypt_key_ctx --output cipher_data plaintext_data
# Input files: rsaencryption_key_ctx, plaintext_data - Output file: cipher_data
```

The command `tpm2 load` is used to load objects into the TPM when both the public and private part are needed for a certain command to be present inside the TPM. Regarding a decryption operation involving a private key, this is required. The decryption process will use a key context (`rsadecrypt_key_ctx`) for referencing the private key inside the TPM. This handle is created by the `tpm2 load` command as follows:

```
tpm2 load --parent-context primary_ctx --public key_pub --private key_priv
 ↳ --key-context rsadecrypt_key_ctx
# Input files: primary_ctx, key_pub, key_priv - Output file:
 ↳ rsadecrypt_key_ctx
```

Additionally to the key context (`rsadecrypt_key_ctx`), the decryption command needs the previously created file with cipher-text data (`cipher_data`) and optionally a file descriptor for the output data (`decrypted_data`) as command line parameters. After executing this command, the content of `decrypted_data` or the terminal output is the same as the `plaintext_data`.

```
tpm2 rsadecrypt --key-context rsadecrypt_key_ctx cipher_data
# Input files: rsadecrypt_key_ctx cipher_data
# Optional output to file: --output decrypted_data
```

The corresponding terminal output should look as follows:

```
$ tpm2 createprimary --hierarchy e --hash-algorithm sha256 --key-algorithm
 ↳ rsa2048 --key-context primary_ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value:
   ↳ fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
rsa: b5602c2838730a108984ab7eb9886f98ec11c2bfe44a300f013baf639897bae1cf8d...
$ tpm2 create --hash-algorithm sha256 --key-algorithm rsa2048
 ↳ --parent-context primary_ctx --public key_pub --private key_priv
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|decrypt|sign
```

```
  raw: 0x60072
type:
  value: rsa
  raw: 0x1
exponent: 65537
bits: 2048
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: null
  raw: 0x10
sym-mode:
  value: (null)
  raw: 0x0
sym-keybits: 0
rsa: cafd9cb68c5279f2010e99f37b69820aed75a3f64e4be759db8f9f69687f0c2e...
$ tpm2 loadexternal --hierarchy n --public key_pub --key-context rsaencrypt_key_ctx
name: 000bdee578e96855a778ae680159aa304ebe56a8cbfe2580c7a7b08278a3cb57204b
$ echo "Hello World" >plaintext_data
$ tpm2 rsaencrypt --key-context rsaencrypt_key_ctx --output cipher_data plaintext_data
$ tpm2 load --parent-context primary_ctx --public key_pub --private key_priv --key-context rsadecrypt_key_ctx
name: 000bdee578e96855a778ae680159aa304ebe56a8cbfe2580c7a7b08278a3cb57204b
$ tpm2 rsadecrypt --key-context rsadecrypt_key_ctx cipher_data
Hello World
```

Sign and Verify using ECC 256

Creating a signature with a private key can be done for any data. The only precondition is that it is possible to create a hash value of the data which is e.g. a file or a message. If that's possible, the output of the hash function will be used as the input for the signing operation. After that, the signature can be used to check both the authenticity and integrity of data which was sent by the owner of the private key and received at a remote location. Therefore, the receiver needs to have the public key to successfully verify the signature. In terms of the TPM the following commands have to be executed to sign a file and verify the signature.

- tpm2 createprimary
- tpm2 create
- tpm2 load
- tpm2 sign
- (tpm2 loadexternal)

- (tpm2 verifysignature)

The first three steps (`createprimary`, `create`, `load`) are basically the same as for the previous section. Both use cases need to have an asymmetric key pair available which can be used for encryption/decryption or signature/verification respectively. The only difference is that the `--key-algorithm` was changed from `rsa` to `ecc` and the name of the context file was changed to `eccsigning_key_ctx`. Note however, that the algorithm of the primary key does not matter in the presented cases, since the child key is encrypted symmetrically inside the TPM. The asymmetric algorithm provided here only servers for encrypted import operations.

```
tpm2 createprimary --hierarchy e --hash-algorithm sha256 --key-algorithm ecc --key-context primary_ctx
# Output file: primary_ctx

tpm2 create --hash-algorithm sha256 --key-algorithm ecc --parent-context primary_ctx --public key_pub --private key_priv
# Input file: primary_ctx - Output files: key_pub, key_priv

tpm2 load --parent-context primary_ctx --public key_pub --private key_priv --key-context eccsigning_key_ctx
# Input files: primary_ctx, key_pub, key_priv - Output file: eccsigning_key_ctx
```

The `tpm2 sign` command makes use of the signing key (`--key-context eccsigning_key_ctx`) and creates a signature which is then saved in the signature file. Before signing the file, a digest from the input_data is created using SHA256.

```
tpm2 sign --key-context eccsigning_key_ctx --signature signature_file input_data
# Input files: eccsigning_key_ctx, input_data - Output files: signature_file
```

Now, the command `tpm2 verifysignature` can be used to verify the signature which was created just before. There is no need to reload the public key into the TPM because it was already loaded with the previous load command. In case there is a reboot of the system or the signature is verified on a different system, then the public key has to be loaded into the TPM before the signature can be verified. This can be done using the command `tpm2 loadexternal` as described in the previous section.

```
tpm2 verifysignature --key-context eccsigning_key_ctx --message input_data --signature signature_file
 # Input files eccsigning_key_ctx, input_data, signature_file
```

The corresponding terminal output should look as follows:

```
$ tpm2 createprimary --hierarchy e --hash-algorithm sha256 --key-algorithm ecc --key-context primary_ctx
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|restricted|decrypt
  raw: 0x30072
type:
  value: ecc
```

```
  raw: 0x23
curve-id:
  value: NIST p256
  raw: 0x3
kdfa-alg:
  value: null
  raw: 0x10
kdfa-halg:
  value: (null)
  raw: 0x0
scheme:
  value: null
  raw: 0x10
scheme-halg:
  value: (null)
  raw: 0x0
sym-alg:
  value: aes
  raw: 0x6
sym-mode:
  value: cfb
  raw: 0x43
sym-keybits: 128
x: eca6c97c4b43231176b043488830632da9a3f7baefaa73898deff6fd896663b5
y: 3033c24be8d34534d519eea7b9bf2baaa8324676af2d8bd4e3de75a817ee9eaa
$ tpm2 create --hash-algorithm sha256 --key-algorithm ecc --parent-context primary_ctx --public key_pub --private key_priv
name-alg:
  value: sha256
  raw: 0xb
attributes:
  value: fixedtpm|fixedparent|sensitivedataorigin|userwithauth|decrypt|sign
  raw: 0x60072
type:
  value: ecc
  raw: 0x23
curve-id:
  value: NIST p256
  raw: 0x3
kdfa-alg:
  value: null
  raw: 0x10
kdfa-halg:
  value: (null)
  raw: 0x0
scheme:
  value: null
  raw: 0x10
scheme-halg:
```

```
  value: (null)
  raw: 0x0
sym-alg:
  value: null
  raw: 0x10
sym-mode:
  value: (null)
  raw: 0x0
sym-keybits: 0
x: 451a818210f3fe9c908e028c1f4cd496d6f5d69499b9a9e868e8daa6188c5110
y: c8419b4ef7083bce1665bc309c08c7d1270139c65b4aabe68e525f10887575c4
$ tpm2 load --parent-context primary_ctx --public key_pub --private key_priv --key-context eccsigning_key_ctx
name: 000bebf1d29ee27c17fe5cbadd30738ed91f595f96f707b09ac4d964347554aecb52
$ tpm2 sign --key-context eccsigning_key_ctx --signature signature_file input_data
$ tpm2 verifysignature --key-context eccsigning_key_ctx --message input_data --signature signature_file
```

7 Using TPM with OpenSSL

The `tpm2-openssl` project[11] implements a provider that can be used to extend OpenSSL 3.0.0 and above with TPM functionality. In order to do so, the `tpm2` provider is loaded into the OpenSSL context to handle certain tasks on behalf of openssl.

Note: For OpenSSL 1.1.1 an alternative implementation is provided with the `tpm2-tss-engine` project[12].

7.1 Installing the software

First, the `tpm2-openssl` software module is installed. Due to a bug in the official debian package, this cannot be used. Instead the module needs to be compiled manually until the bug is resolved.

```
# Currently, this version does not work
#sudo apt --yes install tpm2-openssl

# Instead, install the dependencies of tpm2-openssl
sudo apt --yes install build-essential autoconf autoconf-archive libtool
# Retrieve the latest version of the source code
git clone --depth=1 https://github.com/tpm2-software/tpm2-openssl.git
cd tpm2-openssl
# Build the module
./bootstrap
./configure
make -j
# Install the module
sudo make install
```

[11] https://github.com/tpm2-software/tpm2-openssl
[12] https://github.com/tpm2-software/tpm2-tss-engine

7.2 Interfacing with OpenSSL via command line

When using the command line interface of OpenSSL the provider can be activated by adding the argument `-provider tpm2` to the command line. For some commands, the `provider base` is added additionally. This is needed by OpenSSL in cases where an operation is performed that the tpm2 provider does not support, such as calculating a hash using `SHA2-512`.

To prepare the test scenario, a keypair is generated. The following example uses an ECC P256 keypair. This key is created as a child of the default template for an SRK (a primary key in the storage hierarchy). The corresponding public and private key blobs are encoded as in the `TSS2 PRIVATE KEY` format and encoded in PEM in the file `testkey.priv`. This format is agreed upon and compatible with several OpenSource projects, such as also GNUTLS and OpenConnect.

From this key pair, the public key is exported to the standard `PUBLIC KEY` format that is also usable without a TPM and `tpm2-openssl` module.

```
# Generate ECC P256 key pair as PEM
openssl genpkey -provider tpm2 -algorithm EC -pkeyopt group:P-256 -out testkey.priv
# Output file testkey.priv

# Verify and display the key
openssl ec -provider tpm2 -provider base -in testkey.priv -check -text -noout
# Input file testkey.priv

# Export the public key into separate file for verifier
openssl pkey -provider tpm2 -provider base -in testkey.priv -passin pass: -pubout -out testkey.pub
# Input file testkey.priv - Output file testkey.pub
```

Now the keypair `testkey.priv` can be used for singing a document `testdata`. The signature is stored in the file `testdata.sig`.

```
# Prepare data to be signed
echo -n "abcde12345abcde12345abcde12345ab" > testdata
# Output file testdata

# Sign the data
openssl pkeyutl -provider tpm2 -provider base -sign -inkey testkey.priv -in testdata -out testdata.sig
# Input files testkey.priv, testdata - Output file testdata.sig
```

This signature can then be verified without using the `tpm2-openssl` module using the previously exported public key `testkey.pub`.

```
openssl pkeyutl -verify -pubin -inkey testkey.pub -in testdata -sigfile testdata.sig
# Input files testkey.pub, testdata, testdata.sig
```

The corresponding terminal output should look as follows:

```
$ openssl genpkey -provider tpm2 -algorithm EC -pkeyopt group:P-256 -out testkey.priv
Warning: generating random key material may take a long time
```

```
if the system has a poor entropy source
$ openssl ec -provider tpm2 -provider base -in testkey.priv -check -text -noout
read EC key
Private-Key: (EC P-256, TPM 2.0)
pub:
    04:a4:0c:b5:c8:20:61:88:8e:e6:2e:cc:a2:a1:9a:
    34:43:ad:83:65:59:28:d9:15:69:0e:d9:aa:29:4f:
    bf:0e:fb:df:c0:89:b6:e6:a3:91:52:c8:78:41:c8:
    64:9f:18:3b:3e:96:49:27:1b:53:28:0e:07:14:a4:
    f1:e0:e0:86:c5
ASN1 OID: prime256v1
Object Attributes:
  fixedTPM
  fixedParent
  sensitiveDataOrigin
  userWithAuth
  decrypt
  sign / encrypt
EC Key valid.
$ openssl pkey -provider tpm2 -provider base -in testkey.priv -passin pass: -pubout -out testkey.pub
$ echo -n "abcde12345abcde12345abcde12345ab" > testdata
$ openssl pkeyutl -provider tpm2 -provider base -sign -inkey testkey.priv -in testdata -out testdata.sig
$ openssl pkeyutl -verify -pubin -inkey testkey.pub -in testdata -sigfile testdata.sig
Signature Verified Successfully
```

More examples can be found in the `tpm2-openssl` test suit[13].

7.3 Interfacing with OpenSSL from *NGINX* WebServer

The following shows the usage of the TPM in a WebServer for protecting the long-term identity keys. In this use case the identity key that is used during the handshake of an SSL/TLS communication is created on and stored with the TPM. During the handshake this key is used to provide proof of the identity of the server. During this handshake also the session secrets are negotiated between client and server. These will be held inside the OpenSSL process context at the same isolation level as the data that is being sent of this SSL/TLS channel.

[13] https://github.com/tpm2-software/tpm2-openssl/tree/master/test

Preparing keys and certificates

First, a new keypair and a self-signed certificate are generated. The call to openssl will ask for the information to be written to said certificate. It is possible to press directly on each line in order to use the example values provided in the "`[...]`" brackets.

```
openssl req -provider tpm2 -provider default -propquery '?provider=tpm2' -x509 -new -newkey ec -pkeyopt group:P-256 -keyout httpserver-key.pem -out httpserver-crt.pem
# Output files: httpserver-key.pem, httpserver-crt.pem
```

Installing and Configuring NGINX

First the NGINX WebServer needs to be installed on the system:

```
# Install nginx WebServer
sudo apt --yes install nginx
```

Then the configuration file of NGINX needs to be edited:

```
sudo nano /etc/nginx/sites-enabled/default
# Exit the editor using CTRL+x -> y
```

Inside this file, in the server section, the necessary lines need to be added to enable the SSL/TLS communication (the lines starting with `listen`). Then the lines need to be added that provide the paths to the certificate and keypair to be used for authentication (the lines starting with `ssl_`):

```
server {
        # SSL configuration
        #
        listen 443 ssl default_server;
        listen [::]:443 ssl default_server;
        ssl_certificate /home/pi/httpserver-crt.pem;
        ssl_certificate_key /home/pi/httpserver-key.pem;
```

Also the global configuration for OpenSSL needs to be updated. The reason is to make the `tpm2-openssl` provider module globally available. Only then can the NGINX WebServer correctly parse the `TSS2 PRIVATE KEY` that is provided in the keypair file.

```
sudo nano /etc/ssl/openssl.cnf
# Exit the editor using CTRL+x -> y
```

In the section `openssl_init` the providers section needs to be added. In the provider section, the default provider is configured to remain active, but also the `tpm2-openssl` provider module is declared:

```
[openssl_init]
providers = provider_sect

[provider_sect]
default = default_sect
tpm2privder = tpm2provider_sect
```

```
[default_sect]
activate = 1

[tpm2provider_sect]
activate = 1
module = /usr/lib/aarch64-linux-gnu/ossl-modules/tpm2.so
```

After a restart of NGINX the WebServer should be ready to receive connections:

```
sudo systemctl restart nginx
```

In order to test the status of NGINX the status can be queried from systemd:

```
sudo systemctl status nginx
```

The corresponding terminal outputs should look as follows:

```
$ openssl req -provider tpm2 -provider default -propquery '?provider=tpm2' -x509 -new -newkey ec -pkeyopt group:P-256 -keyout
 httpserver-key.pem -out httpserver-crt.pem
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
$ sudo apt --yes install nginx
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  libpisp0.0.1
Use 'sudo apt autoremove' to remove it.
The following additional packages will be installed:
  nginx-common
Suggested packages:
  fcgiwrap nginx-doc
The following NEW packages will be installed:
  nginx nginx-common
0 upgraded, 2 newly installed, 0 to remove and 1 not upgraded.
Need to get 606 kB of archives.
After this operation, 1,708 kB of additional disk space will be used.
```

```
Get:1 http://deb.debian.org/debian bookworm/main arm64 nginx-common all 1.22.1-9 [112 kB]
Get:2 http://deb.debian.org/debian bookworm/main arm64 nginx arm64 1.22.1-9 [494 kB]
Fetched 606 kB in 0s (2,788 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 148291 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.22.1-9_all.deb ...
Unpacking nginx-common (1.22.1-9) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.22.1-9_arm64.deb ...
Unpacking nginx (1.22.1-9) ...
Setting up nginx-common (1.22.1-9) ...
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service -> /lib/systemd/system/nginx.service.
Setting up nginx (1.22.1-9) ...
Upgrading binary: nginx.
Processing triggers for man-db (2.11.2-2) ...
$ sudo nano /etc/nginx/sites-enabled/default
$ sudo nano /etc/ssl/openssl.cnf
$ sudo systemctl restart nginx
$ sudo systemctl status nginx
- nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Thu 2023-11-16 10:43:24 CET; 7s ago
       Docs: man:nginx(8)
    Process: 3086 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 3089 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 3092 (nginx)
      Tasks: 5 (limit: 3912)
        CPU: 120ms
     CGroup: /system.slice/nginx.service
             |-3092 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             |-3093 "nginx: worker process"
             |-3094 "nginx: worker process"
             |-3095 "nginx: worker process"
             +-3096 "nginx: worker process"

Nov 16 10:43:23 piwithtpm systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 16 10:43:24 piwithtpm systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
```

Testing the connection to NGINX

In order to test the connection to NGINX

```
curl -v --insecure https://localhost
```

The corresponding terminal outputs should look as follows:

```
$ curl -v --insecure https://localhost
*   Trying 127.0.0.1:443...
* Connected to localhost (127.0.0.1) port 443 (#0)
...
* Server certificate:
*  subject: C=AU; ST=Some-State; O=Internet Widgits Pty Ltd
...
<h1>Welcome to nginx!</h1>
...
```

7.4 Interfacing with OpenSSL from *curl* WebClient

It is also possible to perform client authentication in an SSL/TLS connection using the `tpm2-openssl` provider module. In order to test this however, a webserver must offer this option of client authentication to the webclient.

Enabling client certificate authentication in NGINX

In a first step, the nginx webserver needs to allow connection clients to provide a client authentication.

```
sudo nano /etc/nginx/sites-enabled/default
# Exit the editor using CTRL+x -> y
```

When editing this file, make sure that the SSL/TLS options are enabled. In the following listing the values for a TPM enabled web server are still enabled. Further, the line `ssl_verify_client optional_no_ca;` needs to be added to the `server` section.

```
server {
        # SSL configuration
        # ensure working ssl configuration is configured, e.g. the previous
        listen 443 ssl default_server;
        listen [::]:443 ssl default_server;
        ssl_certificate /home/pi/httpserver-crt.pem;
        ssl_certificate_key /home/pi/httpserver-key.pem;

        # Add the following line
        ssl_verify_client optional_no_ca;
```

Finally, NGINX must be restarted to start accepting client certificates.

```
sudo systemctl restart nginx
```

Preparing the client

For the client authentication a keypair as well as a certificate file need to be created, similar to the server side certificate. The content for the certificate that the tool asks for can be filled in with the example values by hitting .

```
openssl req -provider tpm2 -provider default -propquery '?provider=tpm2' -x509 -new -newkey ec -pkeyopt group:P-256  -keyout httpclient-key.pem -out httpclient-crt.pem
# Output files: httpclient-key.pem httpclient-crt.pem
```

Also the file `openssl.cnf` needs to enable the `tpm2-openssl` provider similar to the server side example. In case this was not already done, edit this file accordingly.

```
sudo nano /etc/ssl/openssl.cnf
# Exit the editor using CTRL+x -> y
```

In the section `openssl_init` the providers section needs to be added. In the provider section, the default provider is configured to remain active, but also the `tpm2-openssl` provider module is declared:

```
[openssl_init]
providers = provider_sect

[provider_sect]
default = default_sect
tpm2privder = tpm2provider_sect

[default_sect]
activate = 1

[tpm2provider_sect]
activate = 1
module = /usr/lib/aarch64-linux-gnu/ossl-modules/tpm2.so
```

Testing the communication

In order to test the communication, curl is used again. There will not be an output that indicates the successful transfer of the client certificate though. Therefore, it is possible to enable logging of the TSS by prepending `TSS2_LOG=esys+trace` to the command invocation. There one can observe the communication with the TPM that also includes a call to `Esys_Sign`. During this call, the private key is used for authenticating to the server.

```
curl --insecure -v --key httpclient-key.pem --cert httpclient-crt.pem https://localhost
# Input files: httpclient-key.pem httpclient-crt.pem

# With log information; look for Esys_Sign
TSS2_LOG=esys+trace curl --insecure -v --key httpclient-key.pem --cert httpclient-crt.pem https://localhost
```

The corresponding terminal outputs should look as follows:

```
$ sudo nano /etc/nginx/sites-enabled/default
$ sudo systemctl restart nginx
$ openssl req -provider tpm2 -provider default -propquery '?provider=tpm2' -x509 -new -newkey ec -pkeyopt group:P-256  -keyout httpclient-key.pem -out httpclient-crt.pem
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:
State or Province Name (full name) [Some-State]:
Locality Name (eg, city) []:
Organization Name (eg, company) [Internet Widgits Pty Ltd]:
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:
Email Address []:
$ curl --insecure -v --key httpclient-key.pem --cert httpclient-crt.pem https://localhost
*   Trying 127.0.0.1:443...
* Connected to localhost (127.0.0.1) port 443 (#0)
...
<h1>Welcome to nginx!</h1>
...
```

8 Using TPM with PKCS11

The TPM can also be used as a "virtual smartcard". For this, the project tpm2-pkcs11[14] implements the PKCS11 interface usually used by smart cards.

This module as well a the pkcs11-tool can be installed directly from the Raspberry Pi® OS repositories:

```
sudo apt --yes install libtpm2-pkcs11* opensc
```

In its initial state, the `tpm2-pkcs11` module offers a single uninitialized token slot. This slot can then be initialized in order to create a first token (i.e. virtual smartcard) with a name (i.e. `label`) and a security officer pin (i.e. `so-pin`, the administrator account). In order to offer several virtual smart cards on a single system the `tpm2-pkcs11` module will then also create a new uninitialized token. This means there will always be an uninitialized token present and thus arbitrary numbers of smart cards are supported:

```
# List all tokens
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --list-token-slots

# Initialize the token in slot 0 with label "mytoken" and so-pin "1234"
```

[14] https://github.com/tpm2-software/tpm2-pkcs11

```
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --init-token --label="mytoken" --so-pin="1234"

# List all tokens; including the newly created token
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --list-token-slots
```

After initialization of the token the user pin is initialized. This will be used for all further operations:

```
# Initialize the user pin to "5678" using the previously set so-pin "1234"
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --init-pin --so-pin="1234" --login --pin="5678"
```

In the next step a cryptographic key pair based on ECC P256 is created with the label “mykey”. Initially the list of objects is empty. After the key pair generation the public key is publicly visible. In order to also see the private key, a login needs to be provided:

```
# List objects (empty list)
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects

# Create a keypair labeled "mykey"
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --login --pin="5678" --label="mykey" --keypairgen --usage-sign --key-type EC:prime256v1

# List objects (only public key visible)
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects

# List objects after a login (also private key visible)
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects --login --pin 5678
```

In order to use the new key pair created on the virtual smart card, the public key needs to be exported. Then a file containing testdata is signed. The signature can then be verified using openssl and the exported public key.

```
# Export the public key
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --read-object --slot-index=0 --label="mykey" --type pubkey --output-file pubkey.der
# Output file pubkey.der

# Sign a file
pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --login --pin 5678 --sign --input-file testdata --output-file testdata.sig --mechanism ECDSA --signature-format openssl
# Input file testdata - Output file testdata.sig

# Verify the signature
openssl pkeyutl -verify -inkey pubkey.der -keyform DER -in testdata -sigfile testdata.sig -pubin
```

```
# Input files testdata, testdata.sig
```

The corresponding terminal outputs should look as follows:

```
$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --list-token-slots
Available slots:
Slot 0 (0x1):
  token state:   uninitialized

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --init-token --label="my first token" --so-pin="1234"
Using slot with index 0 (0x1)
Token successfully initialized

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --list-token-slots
Available slots:
Slot 0 (0x1): mytoken
  token label        : mytoken
  token manufacturer : Infineon
  token model        : SLB9672
  token flags        : login required, rng, token initialized, PIN initialized
  hardware version   : 1.59
  firmware version   : 15.22
  serial num         : 0000000000000000
  pin min/max        : 0/128
Slot 1 (0x2):
  token state:   uninitialized

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --init-pin --so-pin="1234" --login --pin="5678"
Using slot with index 0 (0x1)
User PIN successfully initialized

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects
Using slot with index 0 (0x1)

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --login --pin="5678" --label="mykey" --keypairgen --usage-sign --key-type EC:prime256v1
Using slot with index 0 (0x1)
Key pair generated:
Private Key Object; EC
  label:      mykey
  Usage:      decrypt, sign
  Access:     sensitive, always sensitive, never extractable, local
  Allowed mechanisms: ECDSA,ECDSA-SHA1,ECDSA-SHA256,ECDSA-SHA384,ECDSA-SHA512
```

```
Public Key Object; EC  EC_POINT 256 bits
  EC_POINT:   04410461f8f0482ba24d6594bffd40540762c3b48f640bc6294ddd60a...
  EC_PARAMS:  06082a8648ce3d030107
  label:      mykey
  Usage:      encrypt, verify
  Access:     local

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects
Using slot with index 0 (0x1)
Public Key Object; EC  EC_POINT 256 bits
  EC_POINT:   04410461f8f0482ba24d6594bffd40540762c3b48f640bc6294ddd60a...
  EC_PARAMS:  06082a8648ce3d030107
  label:      mykey
  Usage:      encrypt, verify
  Access:     local

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --list-objects --login --pin 5678
Using slot with index 0 (0x1)
Public Key Object; EC  EC_POINT 256 bits
  EC_POINT:   04410461f8f0482ba24d6594bffd40540762c3b48f640bc6294ddd60a...
  EC_PARAMS:  06082a8648ce3d030107
  label:      mykey
  Usage:      encrypt, verify
  Access:     local
Private Key Object; EC
  label:      mykey
  Usage:      decrypt, sign
  Access:     sensitive, always sensitive, never extractable, local
  Allowed mechanisms: ECDSA,ECDSA-SHA1,ECDSA-SHA256,ECDSA-SHA384,ECDSA-SHA512

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --read-object --slot-index=0 --label="mykey" --type pubkey --output-file pubkey.der

$ pkcs11-tool --module /lib/aarch64-linux-gnu/libtpm2_pkcs11.so.1.9.0 --slot-index=0 --login --pin 5678 --sign --input-file testdata --output-file testdata.sig --mechanism ECDSA --signature-format openssl
Using slot with index 0 (0x1)
Using signature algorithm ECDSA

$ openssl pkeyutl -verify -inkey pubkey.der -keyform DER -in testdata -sigfile testdata.sig -pubin
Signature Verified Successfully
```

9 Using TPM for HDD encryption

Firstly, the cryptsetup package needs to be installed. This will be used to create the encrypted drive.

```
sudo apt install cryptsetup
```

For ease of use, this tutorial will not use an actual harddisk (partition) but instead create a virtual one. For this purpose, it creates a file with the size of 50MBytes and maps it to the `/dev/loop0` loopback device. This device will then behave similar to a regular harddisk partition, such as the well-known `/dev/sda3` or similar devices.

```
# Create an empty file of 50*1024*1024 bytes
dd if=/dev/zero of=virtual-disk.img bs=1024k count=50
# Output file virtual-disk.img

# Create a loopback device
sudo losetup /dev/loop0 virtual-disk.img
# Input file virtual-disk.img - Output pseudo-file /dev/loop0
```

Next, the (virtual) harddisk partition is initialized as a cryptographically secured device. First, it is crypto-formated and opened. Then the actual file system is initializes. Finally, the partition is mounted to the path `/virtual-disk/`.

```
# Initialized encrypted device
sudo cryptsetup -y luksFormat /dev/loop0
# Input pseudo-file /dev/loop0

# Open the encrypted device
sudo cryptsetup open /dev/loop0 virtual-disk
# Input pseudo-file /dev/loop0 - Output pseudo-file /dev/mapper/virtual-disk

# Create a file system
sudo mkfs.ext4 /dev/mapper/virtual-disk
# Input pseudo-file /dev/mappter/virtual-disk

# Create a directory for mounting
sudo mkdir /virtual-disk
# Output directoy /virtual-disk

# Mount the filesystem
sudo mount /dev/mapper/virtual-disk /virtual-disk/
# Input pseudo-file /dev/mapper/virtual-disk - Input/Output directory
 /virtual-disk
```

Now, a file can be created with an arbitrary content as demonstration for an encrypted file.

```
# Create a file for testing (exit using CTRL+X -> y)
sudo nano /virtual-disk/testfile
# Output file /virtual-disk/testfile
```

Then, the encrypted disk can be unmounted and the encrypted container is closed.

```
# Unmount the filesystem
sudo umount /virtual-disk
```

```
# Close the encrypted drive
sudo cryptsetup close virtual-disk
# Output pseudo file removed /dev/mapper/virtual-disk
```

Now the TPM support is added as additional thentication slot. The password that was added in the beginning remains valid.

```
# Add TPM support to encrypted container
sudo systemd-cryptenroll --tpm2-device=auto --tpm2-with-pin=true /dev/loop0

# In case this shall be amended, the --wipe-slot=tpm2 parameter can be added
# The --tpm2-with-pin parameter is optional, if a password is required.
```

In order to use the encrypted partition the following set of commands needs to be executed

```
# Open the encrypted device
sudo /usr/lib/systemd/systemd-cryptsetup attach virtual-disk /dev/loop0
# Input pseudo-file /dev/loop0 - Output pseudo-file /dev/mapper/virtual-disk
# Note, we are using systemd-cryptset instead of cryptsetup in this step

# Mount the file system
sudo mount /dev/mapper/virtual-disk /virtual-disk/
# Input pseudo-file /dev/mapper/virtual-disk - Input/Output directory /virtual-disk

# Dump the content of the test file
cat /virtual-disk/testfile
# Input file /virtual-disk/testfile
```

Note: If it was required that the encrypted partition is opened during boot, the necessary information can be added to `/etc/crypttab` and the TSS libraries added to the initial ramdisk.

The expected result from the above command is as follows:

```
$ dd if=/dev/zero of=virtual-disk.img bs=1024k count=50
50+0 records in
50+0 records out
52428800 bytes (52 MB, 50 MiB) copied, 0.298099 s, 176 MB/s

$ sudo losetup /dev/loop0 virtual-disk.img

$ sudo cryptsetup -y luksFormat /dev/loop0

WARNING!
========
This will overwrite data on /dev/loop0 irrevocably.

Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for /home/pi/virtual-disk.img:
Verify passphrase:
```

```
$ sudo cryptsetup open /dev/loop0 virtual-disk
Enter passphrase for /home/pi/virtual-disk.img:

$ sudo mkfs.ext4 /dev/mapper/virtual-disk
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 34816 1k blocks and 8720 inodes
Filesystem UUID: eedacc1e-2f4e-423e-82b0-96728a5e9161
Superblock backups stored on blocks:
        8193, 24577

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

$ sudo mkdir /virtual-disk

$ sudo mount /dev/mapper/virtual-disk /virtual-disk/

$ sudo nano /virtual-disk/testfile

$ sudo umount /virtual-disk

$ sudo cryptsetup close virtual-disk

$ sudo /usr/lib/systemd/systemd-cryptsetup attach virtual-disk /dev/loop0
Please enter LUKS2 token PIN: ***
Selected TPM2 PCRs are not initialized on this system, most likely due to a firmware issue. PCR policy is effectively not enforced. Proceeding anyway.

$ sudo mount /dev/mapper/virtual-disk /virtual-disk/

$ cat /virtual-disk/testfile
This is a testfile

$ sudo umount /virtual-disk

$ sudo /usr/lib/systemd/systemd-cryptsetup detach virtual-disk

$ sudo losetup --detach /dev/loop0
```

10 Appendix

10.1 Troubleshooting

/dev/tpm0 does not appear

For the OPTIGA™ TPM SLB 9673 RPi Evaluation board, a Linux® kernel version >= 6.0 is required. The version can be checked using

```
uname --kernel-release
```

and the kernel can be updated to the latest kernel version using

```
sudo rpi-update
```

TPM Return Code decoding

The TPM return codes are documented in the TPM Library specification Part 2, Section 6.6.3 "TPM_RC Values"[15]. The TSS return codes are documented in the TCG TSS 2.0 Overview and Common Structures Specification, Section 3.4 tss2_-common.h Common Return Codes[16]

There is a Linux® tool that decodes all these errors called tpm2_rc_decode[17] (also includes TSS error codes) that is part of the tpm2-tools[18]

There is also a tool in the Windows store called TPM Return Code Decoder[19] which does not contain the TSS error codes decoding though. Note: Ignore the responses that contain "(1.2)" since these would be the TPM 1.2 returns, only look lines with "(2.0)".

Permission Problems

If you receive an error similar to this:

```
ERROR:tcti:src/tss2-tcti/tctildr-dl.c:254:tctildr_get_default() No standard TCTI could be loaded
ERROR:tcti:src/tss2-tcti/tctildr.c:416:Tss2_TctiLdr_Initialize_Ex() Failed to instantiate TCTI
ERROR: Could not load tcti, got: "(null)"
```

This usually indicates that the current user has no permission to access the TPM.

Please check the permissions of the device file:

```
ls -l /dev/tpm*
```

The result should be

[15] https://trustedcomputinggroup.org/wp-content/uploads/TCG_TPM2_r1p59_Part2_Structures_pub.pdf
[16] https://trustedcomputinggroup.org/wp-content/uploads/TSS_Overview_Common_v1_r10_pub09232021.pdf
[17] https://tpm2-tools.readthedocs.io/en/latest/man/tpm2_rc_decode.1/
[18] https://github.com/tpm2-software/tpm2-tools
[19] https://apps.microsoft.com/store/detail/tpm-return-code-decoder/9NBLGGH4V8NP

```
pi@raspberrypi:~$ ls -l /dev/tpm*
crw-rw---- 1 tss root  10,   224 Aug 26 12:42 /dev/tpm0
crw-rw---- 1 tss tss  236, 65536 Aug 26 12:42 /dev/tpmrm0
pi@raspberrypi:~$
```

If the owner of these devices is `root` then please install the `tpm-udev` package and/or restart the system.

Next check that the current user is a member of the tss group:

```
whoami
id --groups --name [username]
```

The result for a username “pi” should be

```
pi@raspberrypi:~$ whoami
pi
pi@raspberrypi:~$ id --groups --name pi
... tss ...
pi@raspberrypi:~$
```

In order to add a user to the group tss call

```
sudo usermod --append --groups tss <username>
```

Also note that a change of group memberships only becomes effective after logging out and back in again.

Driver Related Problems

Examples: Loading the driver fails, TPM device file /dev/tpm0 does not exist or TPM is not found. Please make sure that:

- The TPM is properly connected.
- Check if your kernel has TPM support by running

```
zgrep --ignore-case tpm /proc/config.gz
```

The output should look like this:

```
CONFIG_HW_RANDOM_TPM=m
CONFIG_TCG_TPM=m
CONFIG_TCG_TIS_SPI=m
```

For the OPTIGA™ TPM SLB 9673 the output further needs to contain

```
CONFIG_TCG_TIS_I2C=m
```

You might need to run `modprobe configs` first to create the `/proc/config.gz` archive. Using the wrong kernel may happen after a software upgrade. In that case you have to install the self-compiled kernel again.

- Check that you are using the correct kernel (it should be your self-built kernel 4.4) by running

```
uname -a #or
cat /proc/version
```

- The driver is loaded. You can try to load it manually `sudo modprobe tpm_tis_spi` in order to find out if the automatic driver load mechanism is not working or if the driver does not run at all.

Login via console

In case there are problems with the communication via SSH, it is possible to connect to the Raspberry Pi® using a serial UART communication. In order to do so, the following two files need to be edited:

```
#/boot/config.txt

[all]
# Enable a linux console
enable_uart=1
```

```
#/boot/cmdline.txt

#ensure the following statements are included
console=serial0,115200 console=tty1
```

Boot logs

In order to retrieve early boot logs of the Raspberry Pi®, it is possible to enable uart as follows:

```
#/boot/config.txt

[all]
# Enable a linux console
enable_uart=1
# Enable boot log messages from ROM code
uart_2ndstage=1
# Disable bluetooth, since it shares PINs with UART
dtoverlay=disable-bt
```

```
#/boot/cmdline.txt
# remove quiet and splash
```

Linux® logs

This section describes some useful hints in case you run into problems.

- Checking kernel messages:

```
dmesg | grep tpm
grep tpm /var/log/messages
grep tpm /var/log/syslog
```

- Checking whether the TPM device driver has been loaded:

```
ls /dev/tpm*
```

- Check the Linux® kernel device tree:

```
find /proc/device-tree/ | grep -i -e tpm -e slb
```

Disable automatic device tree loading

It is possible to disable the automatic configuration of the device tree with the TPM driver information by editing the following file:

```
#/boot/config.txt

[all]
force_eeprom_read=0
```

SPI low-level analysis using libsigrok

The Sigrok project with its `pulse-view` front-end for a large number of logic analysers has a pending contribution that enables analysis of the TPM communication protocol. It can be retrieved from a corresponding pull request[20].

10.2 Support on other Linux® distributions

Ubuntu 20.04

On Ubuntu 20.04 the configuration file is located at /boot/firmware/config.txt.

Also, Ubuntu 20.04 unfortunately does not contain the driver module in its initrd. In order to enable it, run:

```
echo tpm_tis_spi | sudo tee -a /etc/initramfs-tools/modules
sudo update-initramfs -k all -c
```

Enabling Device Tree for SPI

Install the device tree compiler:

```
sudo apt install device-tree-compiler
```

Create a Device Tree with the following content, e.g. `tpm-slb9670.dts`:

```
/dts-v1/;

/ {
        compatible = "brcm,bcm2835h";

        fragment@0 {
                target = <0xffffffff>;

                __overlay__ {
                        status = "okay";
                };
        };
```

[20] https://github.com/sigrokproject/libsigrokdecode/pull/88

```
        fragment@1 {
                target = <0xffffffff>;

                __overlay__ {
                        status = "disabled";
                };
        };

        fragment@2 {
                target = <0xffffffff>;

                __overlay__ {
                        #address-cells = <0x01>;
                        #size-cells = <0x00>;
                        slb9672: slb9670@1 {
                                compatible = "infineon,slb9672", "infineon,slb9670", "tcg,tpm_tis-spi";
                                reg = <0x01>;
                                #address-cells = <0x01>;
                                #size-cells = <0x00>;
                                spi-max-frequency = <0x1e84800>;
                                status = "okay";
                                phandle = <0x01>;
                        };
                };
        };

        __symbols__ {
                slb9670 = "/fragment@2/__overlay__/slb9670@1";
        };

        __fixups__ {
                spi0 = "/fragment@0:target:0\0/fragment@2:target:0";
                spidev1 = "/fragment@1:target:0";
        };
};
```

Compile the device tree

```
dtc -@ -I dts -O dtb -o tpm-slb9670.dtbo tpm-slb9670.dts
```

For testing this dtbo can be loaded directly:

```
sudo dtoverlay ./tpm-slb9670.dtbo
```

Copy the device tree to the boot directory:

```
sudo cp tpm-slb9670.dtbo /boot/overlays
```

Open the file config.txt in an editor.

```
sudo nano /boot/config.txt
```

In this file insert following two lines (slb9670 also works for OPTIGA™ TPM SLB 9672):

```
dtparam=spi=on
dtoverlay=tpm-slb9670
```

Enabling Device Tree for I2C

Install the device tree compiler:

```
sudo apt install device-tree-compiler
```

Create a Device Tree with the following content, e.g. `tpm-slb9670.dts`.

Originally, the following device tree should work and would be recommended when porting this driver onto other platforms, however it cannot be used on a Raspberry Pi® due to a bug in the Raspberry Pi®'s I²C hardware[21].

```
/*
 * Device Tree overlay for the Infineon Trusted Platform Module
 */

/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2835";

    fragment@0 {
        target = <&i2c_arm>;
        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;
            status = "okay";

            slb9673: slb9673@2e {
                compatible = "infineon,slb9673", "tcg,tpm-tis-i2c";
                reg = <0x2e>;
                status = "okay";
            };
        };
    };
};
```

Given a bug in Raspberry Pi®'s clock stretching (see this issue[22]) the I²C master needs to be switched to a GPIO based bit-toggle implementation. As such, the following device tree overlay needs to be used:

```
/*
 * Device Tree overlay for the Infineon Optiga TPM SLB 9673 Trusted
   Platform Module
 * add-on boards, which can be used as a secure key storage and hwrng.
```

[21] https://github.com/raspberrypi/linux/issues/4884
[22] https://github.com/raspberrypi/linux/issues/4884

```
 */

/dts-v1/;
/plugin/;

/ {
    compatible = "brcm,bcm2835", "brcm,bcm2708", "brcm,bcm2709";

    /* Due to issue https://github.com/raspberrypi/linux/issues/4884 the
       hardware I2C needs to be disabled and software I2C enabled */
    fragment@0 {
        target = <&i2c_arm>;
        __overlay__ {
            status = "disabled";
        };
    };

    fragment@1 {
        target-path = "/";
        __overlay__ {
            i2c1: i2c-gpio@1 {
                #address-cells = <1>;
                #size-cells = <0>;
                compatible = "i2c-gpio";
                gpios = <&gpio 2 6>, /* SDA GPIO_OPEN_DRAIN */
                        <&gpio 3 6>; /* CLK GPIO_OPEN_DRAIN */
                clock-frequency = <400000>;
                status = "okay";
            };
        };
    };

    /* Add the TPM */
    fragment@2 {
        target = <&i2c1>;
        __overlay__ {
            #address-cells = <1>;
            #size-cells = <0>;

            slb9673: slb9673@2e {
                compatible = "infineon,slb9673", "tcg,tpm-tis-i2c";
                reg = <0x2e>;
                status = "okay";
            };
        };
    };
};
```

Compile the device tree

```
dtc -@ -I dts -O dtb -o tpm-slb9670.dtbo tpm-slb9670.dts
```

Copy the device tree to the boot directory:

```
sudo cp tpm-slb9670.dtbo /boot/overlays
```

Open the file config.txt in an editor.

```
sudo nano /boot/config.txt
```

In this file insert following two lines (slb9670 also works for OPTIGA™ TPM SLB 9672):

```
dtparam=spi=on
dtoverlay=tpm-slb9670
```

10.3 Compiling Linux® with TPM2 driver (for Raspberry Pi® OS)

This is only necessary for system that do not already have the driver compiled as module or statically.

Installing the build dependencies

```
sudo apt install git bc bison flex libssl-dev make
```

Downloading the Linux® kernel source code

```
# git clone --depth=1 --branch <branch> https://github.com/raspberrypi/linux with <branch> replaced; currently this is rpi-5.15.y
git clone --depth=1 https://github.com/raspberrypi/linux
cd linux
```

Configuring the Linux® kernel

Raspberry Pi® 3, 3+, 4, 400, Zero 2 W in 64 Bit:

```
KERNEL=kernel8
make bcm2711_defconfig
```

Raspberry Pi® 4, 400 in 32 Bit:

```
KERNEL=kernel7l
make bcm2711_defconfig
```

Raspberry Pi® 3, 3+, Zero 2 W in 32 Bit:

```
KERNEL=kernel7
make bcm2709_defconfig
```

Checking the Linux® kernel configuration for OPTIGA™ TPM SLB 9672 and SLx 9670 (SPI)

Check whether the driver is enabled.

```
grep -e CONFIG_TCG_TIS .config
```

This should result in either of these results:

```
CONFIG_TCG_TIS_SPI=m
CONFIG_TCG_TIS_SPI=y
```

If instead the following is returned, please enable the driver in the kernel's configuration:

```
# CONFIG_TCG_TIS_SPI is not set
```

Checking the Linux® kernel configuration for OPTIGA™ TPM SLB 9673 (I^2C)

Check whether the driver is enabled.

```
grep -e CONFIG_TCG_TIS .config
```

This should result in either of these results:

```
CONFIG_TCG_TIS_I2C=m
CONFIG_TCG_TIS_I2C=y
```

If instead the following is returned, please enable the driver in the kernel's configuration:

```
# CONFIG_TCG_TIS_I2C is not set
```

Building the Linux® kernel

For 64-Bit

```
make -j4 Image.gz modules dtbs
```

For 32-Bit

```
make -j4 zImage modules dtbs
```

Installing the Linux® kernel

For 64-Bit

```
sudo make modules_install
sudo cp /boot/$KERNEL.img /boot/$KERNEL-backup.img
sudo cp arch/arm64/boot/Image.gz /boot/$KERNEL.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/
sudo cp arch/arm64/boot/dts/overlays/*.dtb* /boot/overlays/
sudo cp arch/arm64/boot/dts/overlays/README /boot/overlays/
```

For 32-Bit

```
sudo make modules_install
sudo cp /boot/$KERNEL.img /boot/$KERNEL-backup.img
sudo cp arch/arm/boot/zImage /boot/$KERNEL.img
sudo cp arch/arm/boot/dts/*.dtb /boot/
sudo cp arch/arm/boot/dts/overlays/*.dtb* /boot/overlays/
sudo cp arch/arm/boot/dts/overlays/README /boot/overlays/
```

Note: If the newly built kernel will not boot, the old kernel can be chosen by adding one of the following line to the file `config.txt` on the fat32 boot partition:

```
image=kernel8-backup.img
image=kernel7l-backup.img
image=kernel7-backup.img
```

Cross-Compiling

In order to cross-compile, additionally install

```
sudo apt install crossbuild-essential-armhf crossbuild-essential-arm64
```

use the following for 64-Bit:

```
KERNEL=kernel8
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2711_defconfig
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- INSTALL_MOD_PATH=modInst modules_install

#mount the boot and root partitions to /mnt/fat32 and /mnt/ext4
sudo cp mnt/fat32/$KERNEL.img mnt/fat32/$KERNEL-backup.img
sudo cp arch/arm64/boot/Image mnt/fat32/$KERNEL.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb mnt/fat32/
sudo cp arch/arm64/boot/dts/overlays/*.dtb* mnt/fat32/overlays/
sudo cp arch/arm64/boot/dts/overlays/README mnt/fat32/overlays/
sudo cp -ar modInst/* /mnt/ext4
```

use the following for 32-Bit Pi4, 400, CM4:

```
KERNEL=kernel7l
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bcm2711_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- zImage modules dtbs
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=modInst modules_install

#mount the boot and root partitions to /mnt/fat32 and /mnt/ext4
sudo cp mnt/fat32/$KERNEL.img mnt/fat32/$KERNEL-backup.img
sudo cp arch/arm/boot/zImage mnt/fat32/$KERNEL.img
sudo cp arch/arm/boot/dts/*.dtb mnt/fat32/
sudo cp arch/arm/boot/dts/overlays/*.dtb* mnt/fat32/overlays/
sudo cp arch/arm/boot/dts/overlays/README mnt/fat32/overlays/
sudo cp -ar modInst/* /mnt/ext4
```

use the following for 32-Bit Pi2, 3, 3+, Zero 2 W, CM3, CM3+:

```
KERNEL=kernel7
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bcm2709_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- zImage modules dtbs
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=modInst modules_install

#mount the boot and root partitions to /mnt/fat32 and /mnt/ext4
sudo cp mnt/fat32/$KERNEL.img mnt/fat32/$KERNEL-backup.img
sudo cp arch/arm/boot/zImage mnt/fat32/$KERNEL.img
sudo cp arch/arm/boot/dts/*.dtb mnt/fat32/
sudo cp arch/arm/boot/dts/overlays/*.dtb* mnt/fat32/overlays/
sudo cp arch/arm/boot/dts/overlays/README mnt/fat32/overlays/
sudo cp -ar modInst/* /mnt/ext4
```

If you are cross-compiling from WSL you cannot access the ext4 file system yet. You should therefore add the modules into a tar archive and transfer them via the /boot partition:

```
(cd modInst/lib/modules && tar cjf ../../../modInst.tar.bz2 *)
```

Then copy the `$KERNEL.img` all `*.dtb*` files and `modInst.tar.bz2` to a subfolder of the `/boot` mount (we will call this folder `kernelUpdate`). On the Raspberry Pi® copy everything to the target locations:

```
KERNEL=kernel8 # or KERNEL=kernel7l or KERNEL=kernel7

(cd /lib/modules && sudo tar xjf /boot/kernelUpdate/modInst.tar.bz2 && sudo chown -R root:root .)

sudo cp /boot/$KERNEL.img /boot/$KERNEL-backup.img
sudo cp /boot/kernelUpdate/$KERNEL.img /boot/$KERNEL.img
sudo cp /boot/kernelUpdate/*.dtb /boot/
sudo cp /boot/kernelUpdate/overlays/*.dtb* /boot/overlays/
sudo cp /boot/kernelUpdate/overlays/README /boot/overlays/
```

10.4 Compiling the TSS from source

Compiling tpm2-tss from source

The latest version of the tpm2-tss (the project for the libtss2-* packages) can be found here[23]. This package provides a middleware for interaction with the TPM based on a C API. Documentation can be found on readthedocs[24].

[23] https://github.com/tpm2-software/tpm2-tss
[24] https://tpm2-tss.readthedocs.io

Compiling tpm2-tools from source

The latest version of the tpm2-tools can be found here[25]. This package provides a set of command line utilities for interacting with the TPM via the tpm2-tss. Documentation can be found on readthedocs[26].

Compiling TPM support for OpenSSL from source

TPM support for OpenSSL can be found here for 1.1.x[27] and here for 3.x.x[28]. These packages provide an OpenSSL engine respectively OpenSSL provider implementation that utilize the TPM.

10.5 Accessing the TPM with eltt2

The Embedded Linux TPM Toolbox 2 (ELTT2)[29] is an Open-Source tool provided by Infineon to enable basic interaction with the TPM.

You can use is as follows:

```
1 git clone --depth=1 https://github.com/Infineon/eltt2.git
2 cd eltt2
3 make
4 sudo ./eltt2 -g
```

For further information and other commands, please refer to the eltt2's README[30].

10.6 Working with the Raspberry Pi®

For user interaction with the Raspberry Pi®, either a monitor and keyboard can be attached directly, or a remote connection via SSH can be used. This section describes both possibilities.

Direct interaction

In order to directly interact with the Raspberry Pi®, attach a USB keyboard to one of the USB ports of the Raspberry Pi® and a monitor via HDMI. This enables the user to view the complete boot process of the Raspberry Pi®.

[25] https://github.com/tpm2-software/tpm2-tools
[26] https://tpm2-tools.readthedocs.io
[27] https://github.com/tpm2-software/tpm2-tss-engine
[28] https://github.com/tpm2-software/tpm2-openssl
[29] https://github.com/Infineon/eltt2
[30] https://github.com/Infineon/eltt2#2-usage-of-eltt2

Connecting via SSH

To connect via Ethernet to the Raspberry Pi®, you can use the remote terminal SSH. Additionally, with SSH you can open multiple simultaneous terminal sessions with the same or different users. SSH should be active by default. In case you have disabled SSH, you can reinstall it by running the following command from a terminal directly with an attached keyboard and monitor (see section 6.8.1):

```
apt-get install ssh
```

In order to avoid warnings regarding missing system locales (that is if your PC uses locales that are not installed on the Raspberry Pi® 3), it is also recommended to disable the corresponding SSH option. To do so, uncomment the corresponding SSH option in the file /etc/ssh/ssh_config on the developer PC:

```
nano /etc/ssh/ssh_config
```

Now change the line with

```
SendEnv LANG LC_*
```

to

```
# SendEnv LANG LC_*
```

To use SSH attach both the developer PC and the Raspberry Pi® to the same network. In case you have a DHCP server running in your LAN, the boards will automatically obtain an IP address on boot, otherwise you have to manually configure an IP address for both of them.

For SSH to work, the IP addresses of both the developer PC and the Raspberry Pi® must be in the same subnet, that is for IPv4 the first three blocks must be the same. For example, 192.168.100.1 and 192.168.100.2 are in the same subnet while 192.168.101.3 is in a different subnet. Normally, the DHCP server in your network (DHCP server can be a dedicated server or run on your router) takes care of this, but in case you configure IP addresses manually, you have to keep that in mind.

Depending on your network configuration, the Raspberry Pi® will also publish its host name in the LAN, which you can get by executing the command hostname on the Raspberry Pi®. In this case, you can use SSH by executing the following command from a terminal on the developer PC:

```
ssh [username]@[hostname]
```

On a Raspberry Pi® the default user is `pi`, the default hostname is `raspberrypi` and the default password for the user `pi` is `raspberry`. Thus, the SSH command would be the following:

```
ssh pi@raspberrypi
```

If you cannot open a connection or your network configuration does not support to connect via the host name, you can also use the IP address, which you can get by executing ifconfig on the Raspberry Pi®. Now you can use SSH by executing the following command from a terminal on the developer PC:

```
ssh [user name]@[IP Address]
```

You can disconnect SSH by executing the following command in the same terminal on the developer PC:

```
exit
```

Exchanging Files

This section describes how to exchange files between the Raspberry Pi® and the developer PC.

Direct Copy on microSD-Card

You can insert the microSD-Card into a card reader attached to the developer PC. Ubuntu® will then automatically mount the partitions on the card and you can copy files to it. Mounting the Raspberry Pi® 3's micro-SD-card works reliably only on Linux-based operating systems, since other operating systems may only be able to mount the ext4 file system of the partitions on the microSD-card and / or may only be able to mount more than one partition on a microSD-card.

Copying via SCP

SCP uses SSH to copy files between two machines in a network. It requires a working SSH client installation on the local machine and a running SSH server on the remote machine. The syntax is as follows:

```
scp [source] [target]
```

`[source]` and `[target]` can either be a normal file path (only in case of files on the local machine) or must have the following form (for files on the remote machine): `[remote username]@[remote hostname or IP address]:[remote file path]`

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition February 29, 2024
Published by
Infineon Technologies AG
81726 Munich, Germany

© 2022 Infineon Technologies AG.
All Rights Reserved.

Do you have a question about this document?
Email: dsscustomerservice@infineon.com

IMPORTANT NOTICE
The information contained in this User Guide is given as a hint for the implementation of the product only and shall in no event be regarded as a description or warranty of a certain functionality, condition or quality of the product. Before implementation of the product, the recipient of this User Guide must verify any function and other technical information given herein in the real application. Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind (including without limitation warranties of non-infringement of intellectual property rights of any third party) with respect to any and all information given in this User Guide.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS
Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.